

A Singularly Useful Representation

Knowing the possible?

We are going to consider some general engineering problems which benefit from one of the most numerically rock solid representations of data known

Knowing the possible?



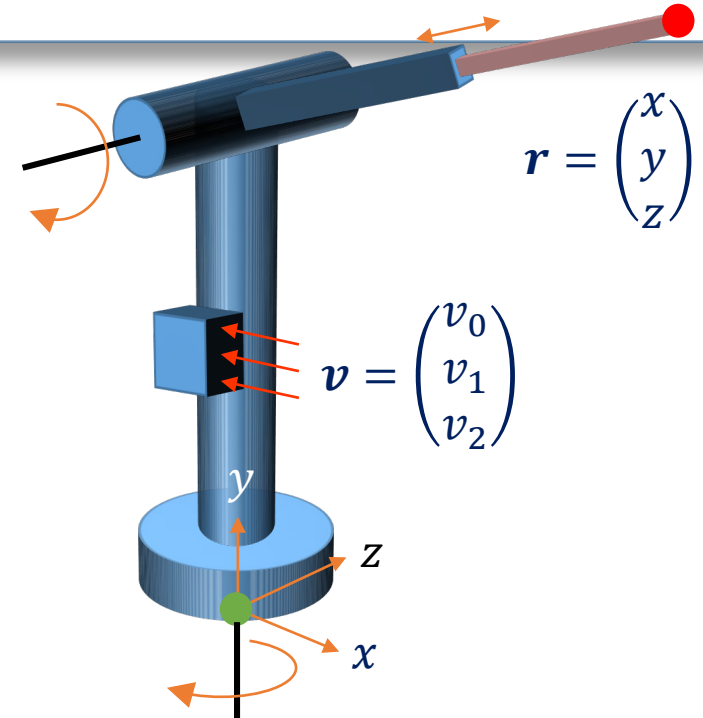
Example: We are designing the control software for a robotic arm. In the 3D world we place the spatial coordinates of points in 3 element column vectors $\mathbf{r} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$. We aim to place the ● anywhere in space which requires controlling 3 values, x , y and z , so surely we must have 3 control *voltages*, $\mathbf{v}$

We have lost the manual!

We do some experiments, setting random voltage vectors $\mathbf{v}_a, \mathbf{v}_b, \mathbf{v}_c, \mathbf{v}_d \dots$ and record the corresponding arm positions $\mathbf{r}_a, \mathbf{r}_b, \mathbf{r}_c, \mathbf{r}_d \dots$

We are confident that the system is *linear*

How can we use this information to generally decide what voltages must be applied to reach a particular location in space?



Knowing the possible?



Example cont: If the results of the experiments were

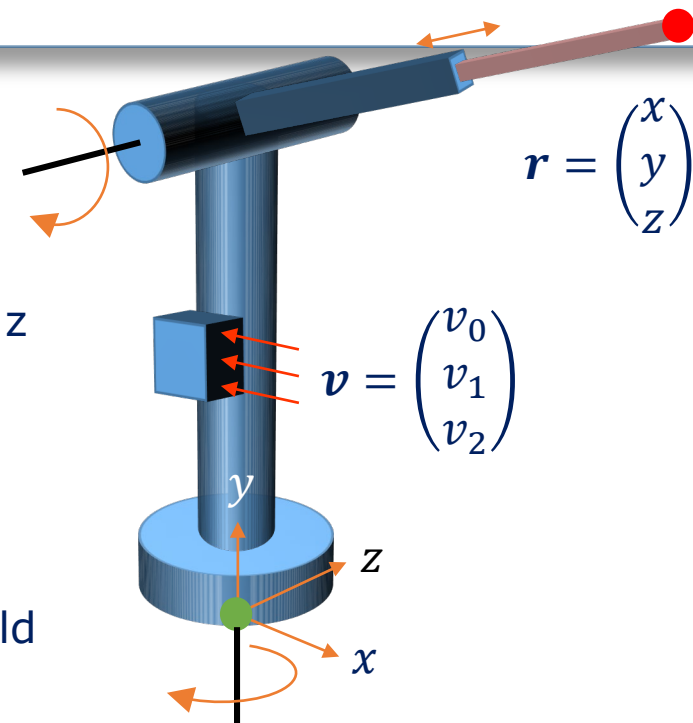
$$r_a = \begin{pmatrix} 2 \\ 1 \\ 0 \end{pmatrix} \quad r_b = \begin{pmatrix} 3 \\ -1 \\ 0 \end{pmatrix} \quad r_c = \begin{pmatrix} 1 \\ 5 \\ 0 \end{pmatrix} \quad r_d = \begin{pmatrix} 3 \\ 3 \\ 0 \end{pmatrix} \quad r_e = \begin{pmatrix} 7 \\ 1 \\ 0 \end{pmatrix} \quad r_f = \begin{pmatrix} 4 \\ 2 \\ 0 \end{pmatrix}$$

What do we notice?

Although we appeared to have 3 control voltages
we never seem to be able to change the value of z

Maybe the bottom axis motor is broken.

The serious point is that it was lucky that we could
spot this fact ourselves in the data



What if it were the extending arm transducer that was broken?

Example cont: If the results of the experiments were

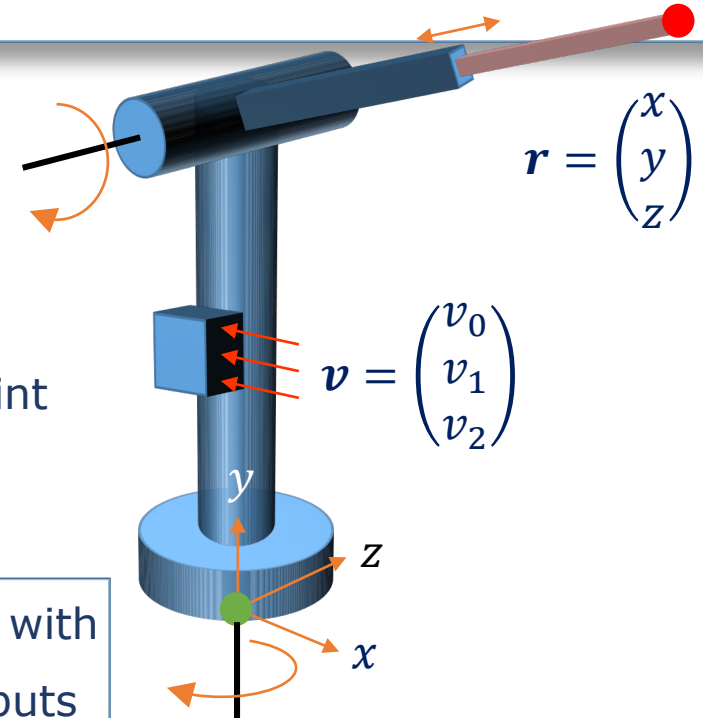
$$r_a = \begin{pmatrix} 2 \\ 1 \\ 4.47 \end{pmatrix} \quad r_b = \begin{pmatrix} 3 \\ 3.83 \\ -1 \end{pmatrix} \quad r_c = \begin{pmatrix} 5 \\ 0 \\ 0 \end{pmatrix} \quad r_d = \begin{pmatrix} 4.98 \\ -0.1 \\ 0.4 \end{pmatrix} \quad r_e = \begin{pmatrix} 4.57 \\ 2 \\ 0.3 \end{pmatrix} \quad r_f = \begin{pmatrix} 4.11 \\ 1.67 \\ 2.30 \end{pmatrix}$$

Is it so obvious?

(Note all have $|r|_2 = 25$)

Physically, the arm can only be located at any point on a sphere of radius 5

This shows us that we need to concern ourselves with the **range** of outputs we can access using our inputs



How do we represent our experiments in matrix form?

Knowing the possible?

How do we represent our experiments in matrix form? Remember how we expressed a superposition on slide ??. A general input can be expressed as

$$\begin{pmatrix} v_0 \\ v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} v_{a,0} \\ v_{a,1} \\ v_{a,2} \end{pmatrix} c_a + \begin{pmatrix} v_{b,0} \\ v_{b,1} \\ v_{b,2} \end{pmatrix} c_b + \begin{pmatrix} v_{c,0} \\ v_{c,1} \\ v_{c,2} \end{pmatrix} c_c \dots$$

$$\begin{pmatrix} v_0 \\ v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} v_{a,0} & v_{b,0} & v_{c,0} & \dots \\ v_{a,1} & v_{b,1} & v_{c,1} & \dots \\ v_{a,2} & v_{b,2} & v_{c,2} & \dots \end{pmatrix} \begin{pmatrix} c_a \\ c_b \\ c_c \\ \dots \end{pmatrix}$$

$$\mathbf{v} = \sum_{i=0}^{N-1} v_i c_i$$

$$\mathbf{v} = \mathbf{V}\mathbf{c}$$

And the corresponding positions

$$\begin{pmatrix} r_x \\ r_y \\ r_z \end{pmatrix} = \begin{pmatrix} r_{a,x} \\ r_{a,y} \\ r_{a,z} \end{pmatrix} c_a + \begin{pmatrix} r_{b,x} \\ r_{b,y} \\ r_{b,z} \end{pmatrix} c_b + \begin{pmatrix} r_{c,x} \\ r_{c,y} \\ r_{c,z} \end{pmatrix} c_c \dots$$

$$\begin{pmatrix} r_x \\ r_y \\ r_z \end{pmatrix} = \begin{pmatrix} r_{a,x} & r_{b,x} & r_{c,x} & \dots \\ r_{a,y} & r_{b,y} & r_{c,y} & \dots \\ r_{a,z} & r_{b,z} & r_{c,z} & \dots \end{pmatrix} \begin{pmatrix} c_a \\ c_b \\ c_c \\ \dots \end{pmatrix}$$

$$\mathbf{r} = \sum_{i=0}^{N-1} r_i c_i$$

$$\mathbf{r} = \mathbf{R}\mathbf{c}$$

Knowing the possible?

It's helpful now to visually emphasis the dimensions of the matrices (e.g. are they square or not)

$$\begin{pmatrix} v \end{pmatrix} = \begin{pmatrix} V \end{pmatrix} \begin{pmatrix} c \end{pmatrix}$$

$$\begin{pmatrix} r \end{pmatrix} = \begin{pmatrix} R \end{pmatrix} \begin{pmatrix} c \end{pmatrix}$$

Control task: What v to apply in order to get a desired r ?

Can we could write $\begin{pmatrix} c \end{pmatrix} = R^{-1} \begin{pmatrix} r \end{pmatrix}$ hence $\begin{pmatrix} v \end{pmatrix} = VR^{-1} \begin{pmatrix} r \end{pmatrix}$?

No because the R matrix is not square and so has no unique inverse

This is a class of engineering challenge facing us and which can be approached using the core theme of today's discussion

But before the maths, another apparently different class of engineering challenges that we will also be able address after today

Knowing too much!

The rate at which the world is generating *data* is incredible

Don't you just feel swamped with unnecessary detail sometimes?

Communications networks; bandwidth = data rate = £

Data storage centres; memory = energy consumption = £ and other topical issues

Example: Here's one to warm the hearts of all academics. *Student says, great course with lots of interesting material - but what's the essential stuff I need to study for the exam?*

Example: This sound system has a large number of settings to allow ultra-fine control of the listening experience. Wonderful, but how do I simply play a CD?

The theme here is;

- *Extracting the most important information*
- *Data compression*
- *Complexity reduction*

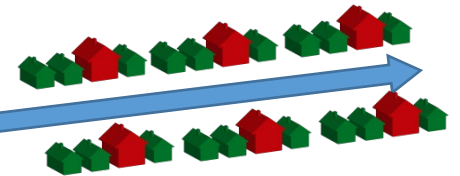
Knowing too much!



Example: A delivery company is route planning for the purpose of optimising fuel use. They have the GPS coordinates of all the destinations. A detailed route is produced spanning many pages and given to the driver. The driver says, yes, but *roughly speaking* where am I going?



As a human being, I might say that the general direction of travel or else the *most significant feature* of the route *for the purpose of evaluating the fuel* is



Complexity Reduction task: How do we separate all the raw data into the *most* and *least* significant aspects, or the *big picture* and the *detail*?

Knowing too much!

How do we represent such a scenario using matrices?

If the i^{th} destination to be visited has 2D coordinates $\begin{pmatrix} r_{i,x} \\ r_{i,y} \end{pmatrix}$, we could consolidate all the destination data into one rectangular matrix

$$R = \begin{pmatrix} r_{0,x} & r_{1,x} & r_{2,x} & \dots \\ r_{0,y} & r_{1,y} & r_{2,y} & \dots \end{pmatrix}$$

Again, depending on the actual data, we may be able to visually spot what we want

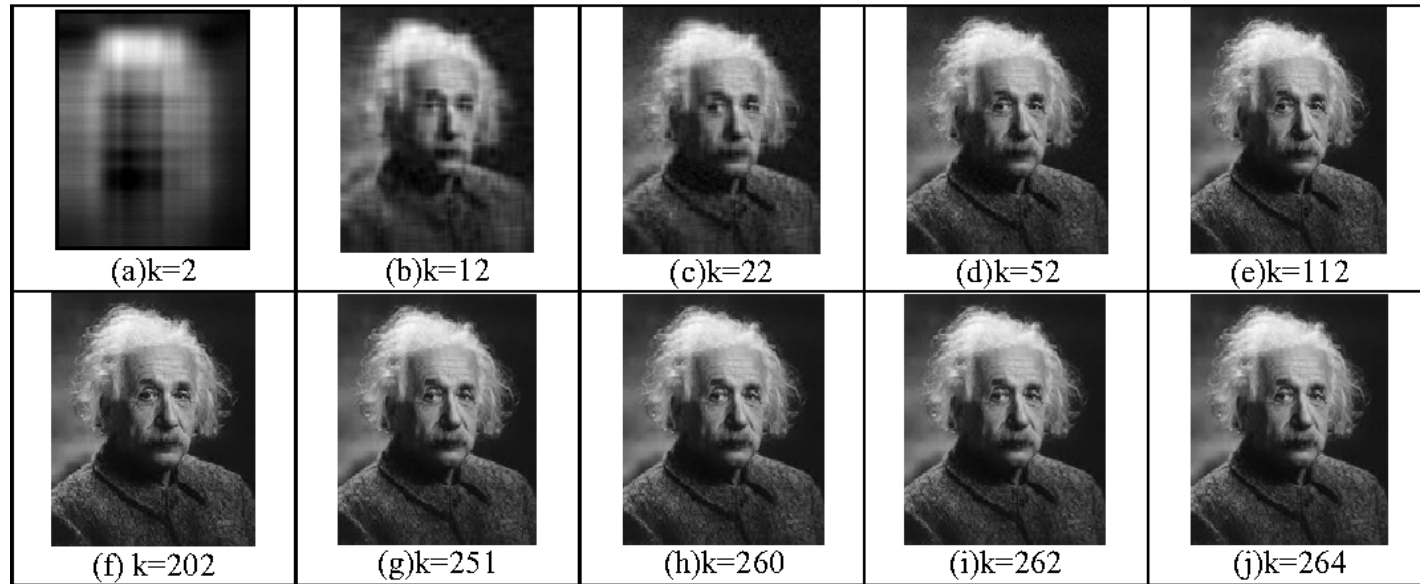
$$R = \begin{pmatrix} 9.8 & 10.1 & 9.7 & 10.2 \\ 2.1 & 2.1 & 1.8 & 1.9 \end{pmatrix}$$

Here I realise that the driver needs to head off toward roughly $\begin{pmatrix} 10 \\ 2 \end{pmatrix}$ and then do some detailed manoeuvres within a small locality of that point

How do we generally extract the most and least significant aspects of the data?

Knowing too much!

Image compression is a related engineering task



Lossy image compression using SVD coding algorithm, K Monisha Aishwarya et al

Published in International Conference on Wireless...2016

DOI:10.1109/WiSPNET.2016.7566363

- *How do we save "memory" with the least impact on perceived quality?*
- *Separate the Important from less important aspects of the image?*
- *Reduce the "complexity" of the image/data?*

It can be shown that: Any matrix, A , can be decomposed (c.f. factorised) into the form $A = Q\sigma P^T$ where Q and P are orthogonal matrices and σ is a diagonal matrix. $Q\sigma P^T$ is the **Singular Value Decomposition (SVD)** of A

Now you may be expecting my usual *except when it can't be* comment here

But no!

The SVD is valid for *all* matrices; square, rectangular, with or without symmetry, real and complex etc; I repeat:

The SVD is valid for *all* matrices

So where's the catch and anyway, why is the SVD so important?

Why is the SVD so important?

Obviously the SVD will be able to solve the classes of motivating examples I described - as we shall show later

- It generally "cleans up noisy data"
- It generally "prioritises data as a sequence of more and less important aspects"
- It resolves many numerical computation problems - for example if a matrix solve problem is on the "borderline" because of finite precision issues
- It is a perfect means of finding the optimum best fit solution to over specified problems
- It tells you "how much control" you really have and "what range of outputs" you can hope to reach with the "available inputs"

Worth a quick look then?

The Singular Value Decomposition

What's the catch?

Previously in our discussions, we have said that *in principle* something can be done, but *in practice*, numerical rounding issues undermine its practical use

The Singular Value Decomposition of a matrix can be practically evaluated with rock solid reliability on real computers

The one negative point about the SVD is that it is *slow* to evaluate. However as computer power is improving the circumstances in which this *cost* is justified by the application is ever increasing.

In summary: The SVD is highly useful, highly reliable calculation, but can be expensive to evaluate.

Let us now look at what is involved in the SVD; As usual, this course does not extend to discussing how we evaluate it

The Singular Value Decomposition

$$A = Q\sigma P^T$$

$$\begin{pmatrix} A \end{pmatrix}^{N \times M} = \begin{pmatrix} Q \end{pmatrix}^{N \times N} \begin{pmatrix} \sigma_0 & & \\ & \sigma_1 & \\ & & \dots \end{pmatrix}^{N \times M} \begin{pmatrix} P^T \end{pmatrix}^{M \times M}$$

$$\begin{pmatrix} A \end{pmatrix}^{N \times M} = \begin{pmatrix} Q \end{pmatrix}^{N \times N} \begin{pmatrix} \sigma_0 & & \\ & \sigma_1 & \\ & & \dots \end{pmatrix}^{N \times M} \begin{pmatrix} P^T \end{pmatrix}^{M \times M}$$

Note the *shapes*, Q and P^T are always square. The "diagonal" matrix is not necessarily square

By the way, this method of showing matrices "pictorially" to clarify relative sizes is often called tableau form

The Singular Value Decomposition

The reason for the power of the SVD is its use of orthogonal matrices

which we previously defined for the differently sized $\mathbf{P}^{M \times M}$ and $\mathbf{Q}^{N \times N}$ as

$$\mathbf{P}^T \mathbf{P} = \mathbf{P} \mathbf{P}^T = \mathbf{I}^{M \times M} \qquad \mathbf{Q}^T \mathbf{Q} = \mathbf{Q} \mathbf{Q}^T = \mathbf{I}^{N \times N}$$

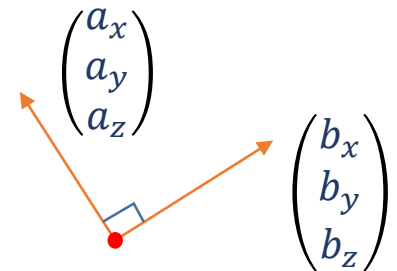
and we said, just for orthogonal matrices, that $\mathbf{P}^T = \mathbf{P}^{-1}$,

Now is the time to put a picture on this which helps apply SVD to real engineering problems

The word *orthogonal* is the higher dimension abstraction of the concept of *perpendicular* in 2D and 3D geometry

Example: Two geometrical vectors $\mathbf{a} = \begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix}$ and $\mathbf{b} = \begin{pmatrix} b_x \\ b_y \\ b_z \end{pmatrix}$ are perpendicular if $a_x b_x + a_y b_y + a_z b_z = 0$

which written in matrix form is $\mathbf{a}^T \mathbf{b} = 0$



Definition of orthogonality between a pair of column vectors of the same size N

$$\mathbf{a}^T \mathbf{b} = 0 \text{ for } \mathbf{a} \neq \mathbf{b}$$

Orthogonality is just N-dimensional perpendicularity

Above we used 3 element columns to represent 3 spatial coordinate values, x, y, z

For the robotic example, the data *actually is* spatial coordinate values

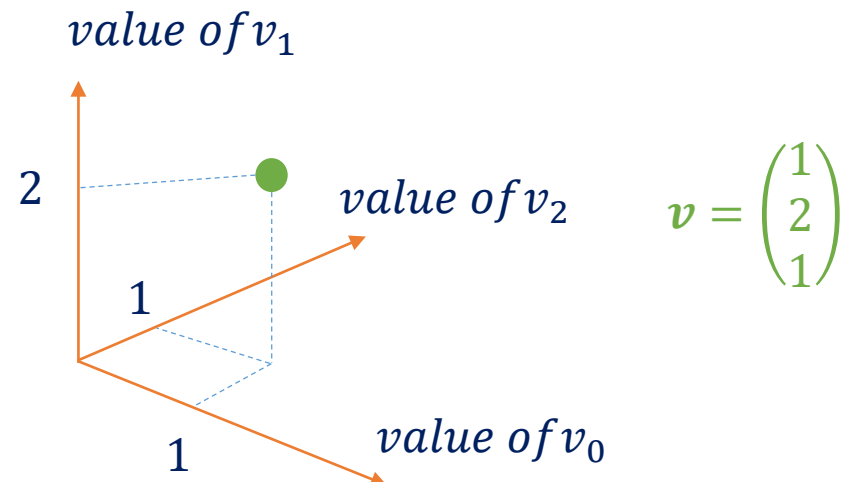
However, there is no reason why, if it helps understanding, that we could *visualise* any 3 values as if they *were* spatial coordinate values

Example: We used a column to represent 3 control voltages $v = \begin{pmatrix} v_0 \\ v_1 \\ v_2 \end{pmatrix}$

We can imagine a plot, where we show each particular v as a point in a 3D space

Why does this help?

Geometry is more intuitive than
abstract matrix theory



Geometrical Interpretation of Column Vector Data



The University of
Nottingham

UNITED KINGDOM • CHINA • MALAYSIA

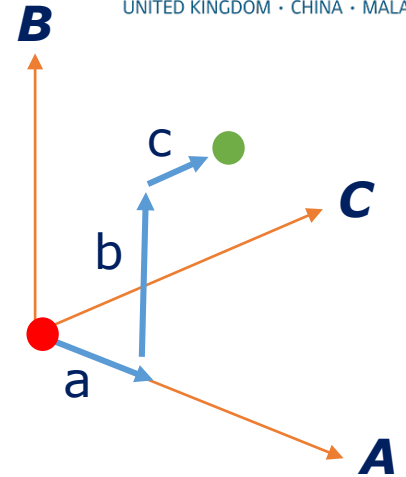
Think of this as plot as saying,

To get the destination point from the origin:

*Travel a distance of "a" in the direction "**A**" then*

*Travel a distance of "b" in the direction "**B**" then*

*Travel a distance of "c" in the direction "**C**" then*



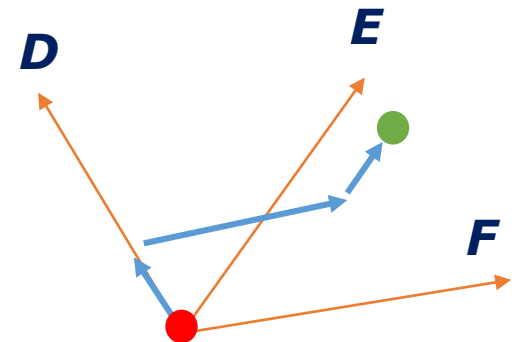
This is expressing the destination as "so much **A**" + "so much **B**" + "so much **C**"

We appreciate that with this approach to specifying the destination we could draw the "axis vectors" in any orientation we choose

This is expressing the **same** destination as

"so much **D**" + "so much **E**" + "so much **F**"

Then ...

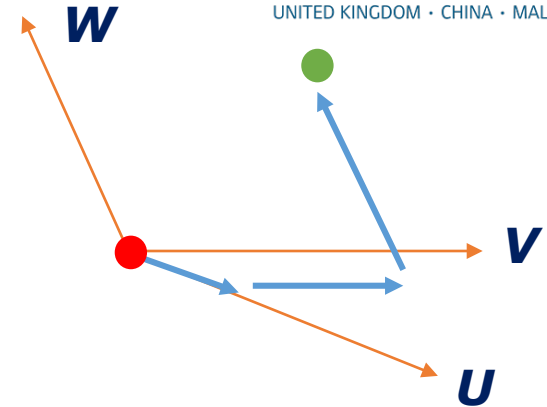


Geometrical Interpretation of Column Vector Data



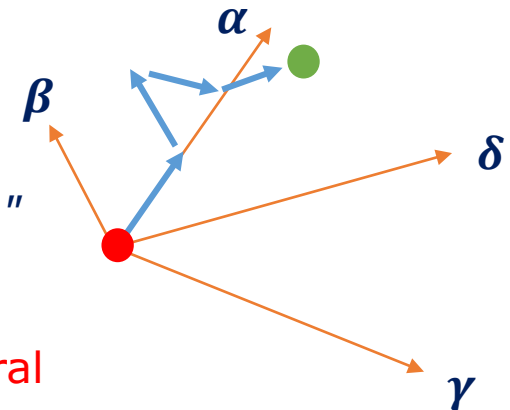
The "components" don't actually need to be perpendicular

This is expressing the destination as
"so much **U**" + "so much **V**" + "so much **W**"



and we can have as many contributing components as we want

This is expressing the destination as
"so much α " + "so much β " + "so much γ " + "so much δ "



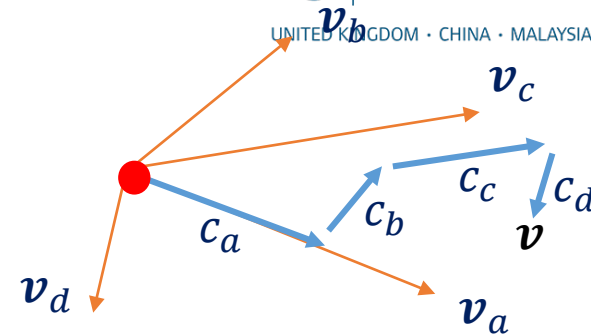
Now compare this with the earlier suggestion that a general control voltage could be expressed as

$$\begin{pmatrix} v_0 \\ v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} v_{a,0} & v_{b,0} & v_{c,0} & v_{d,0} \\ v_{a,1} & v_{b,1} & v_{c,1} & v_{d,1} \\ v_{a,2} & v_{b,2} & v_{c,2} & v_{d,2} \end{pmatrix} \begin{pmatrix} c_a \\ c_b \\ c_c \\ c_d \end{pmatrix} \quad \text{isn't this the same idea ?}$$

$$\begin{pmatrix} v_0 \\ v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} v_{a,0} & v_{b,0} & v_{c,0} & v_{d,0} \\ v_{a,1} & v_{b,1} & v_{c,1} & v_{d,1} \\ v_{a,2} & v_{b,2} & v_{c,2} & v_{d,2} \end{pmatrix} \begin{pmatrix} c_a \\ c_b \\ c_c \\ c_d \end{pmatrix}$$

The total voltage column is being expressed as

"so much v_a " + "so much v_b " + "so much v_c " + "so much v_d "



Why does this help? It is just a picture - we don't need it, but it does allow us to use familiar geometrical concepts to visualise abstract matrix operations

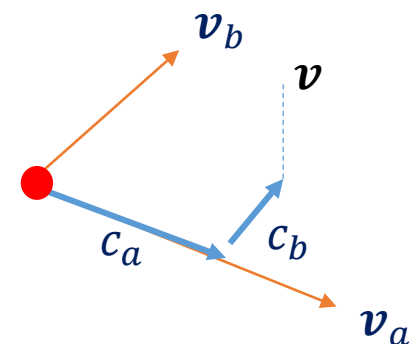
The proper name for the set of "axes" used for a representation is the

Basis Vectors

Example: We *know* that you need exactly 3 axes directions to uniquely identify any spatial point, yet note from the figure above, we can now visualise how one that uses more than 3 contributing components is achieving this.

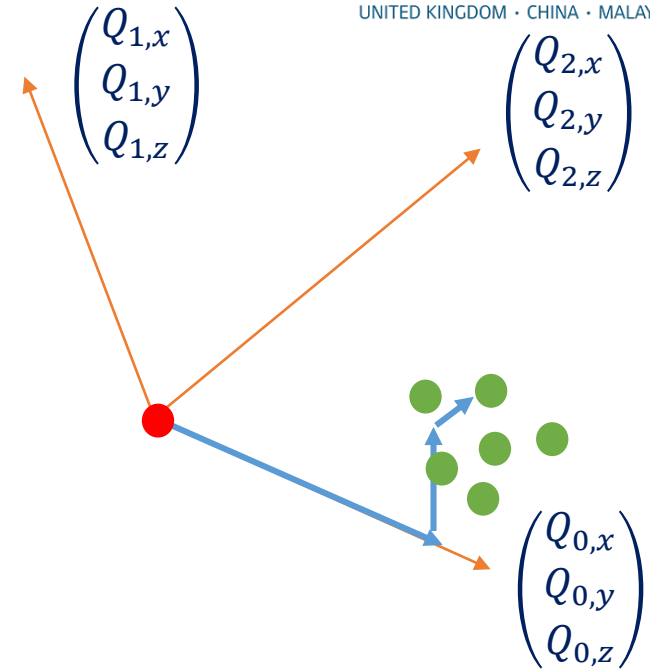
Example: If we only had 1 or 2 axes directions (basis vectors) we easily appreciate from the geometrical picture that we *cannot reach* any destination we want to

See the Control example later



What is SVD doing in this regard?

SVD says: Based on the target data you have given me (the green dots), I recommend that you use the 3 axes Q_0 , Q_1 and Q_2 to describe the location of points in space



It makes this recommendation such that

- The number of axes is no more, or no less, than necessary
- The *big distances* to get to target data will be along the first axis, the *medium distances* along the second axis and the *smallest distances* along the last axis
- The axes used by SVD are *perpendicular*

Let us start cross-checking the value of this with our earlier case studies

The number of axes is no less or no more than necessary

Example: In the robotic arm example, we made multiple (N) test measurements of control voltages and resulting position

$$\begin{pmatrix} \mathbf{r} \end{pmatrix}^{3 \times 1} = \begin{pmatrix} \mathbf{R} \end{pmatrix}^{3 \times N} \begin{pmatrix} \mathbf{c} \end{pmatrix}^{N \times 1}$$

SVD says: You have given me many many test values but basically, you only have a 3D space - so you only really *need* to specify the amounts of these 3 vectors, $\mathbf{Q}_0$, $\mathbf{Q}_1$ and $\mathbf{Q}_2$, to *be able to scan to same overall space*

$$\begin{pmatrix} \mathbf{r} \end{pmatrix}^{3 \times 1} = \begin{pmatrix} \mathbf{Q} \end{pmatrix}^{3 \times 3} \begin{pmatrix} \end{pmatrix}^{3 \times 1}$$

Amount of each axis

*So SVD has extracted out the **simplest** possible way of representing any point in space: Removing unnecessary **redundancy** from the representation*

The *big distances* to get to target data will be along the first axis, the *medium distances* along the second axis and the *smallest distances* along the last axis

This is the role of the σ_i values; These are called the **singular values**.

σ_0 is a measure of the distance one has to travel in the direction of axis Q_0 in order reach the target data given to the SVD

$$\begin{pmatrix} A \end{pmatrix}^{N \times M} = \begin{pmatrix} Q \end{pmatrix}^{N \times N} \begin{pmatrix} \sigma_0 & & \\ & \sigma_1 & \\ & & \dots \end{pmatrix}^{N \times N} \begin{pmatrix} P^T \end{pmatrix}^{M \times M}$$

SVD assures us that: $\sigma_0 > \sigma_1 > \sigma_2 \dots$ Hence, to get to the target data the biggest distances are travelled in the direction of Q_0 and the least distances in the direction of Q_{N-1}

Let's see how that fits with our case studies ...

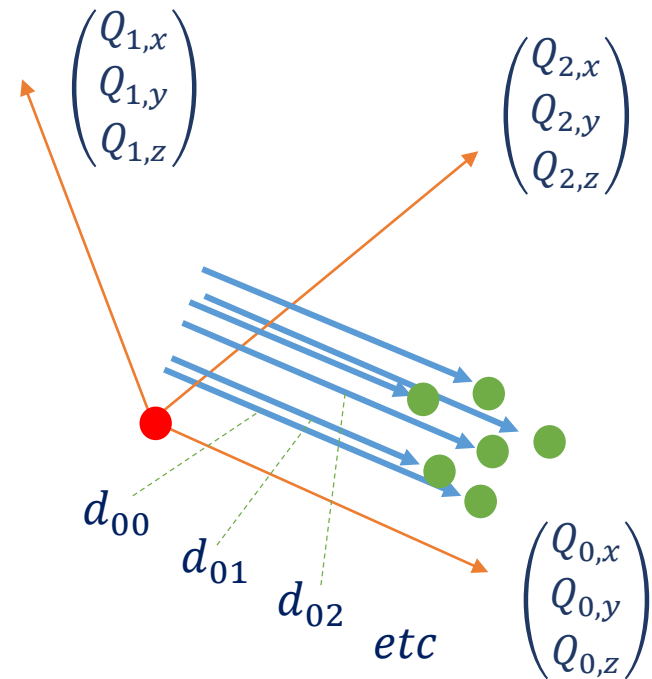
The specific value of each singular value is related to an RMS value

Each singular value is the square root of the sum of the square distances along the corresponding axis to reach the given data set

$$\sigma_i = \sqrt{\sum_{k=0}^{M-1} d_{ik}^2}$$

Note the connection to an RMS value

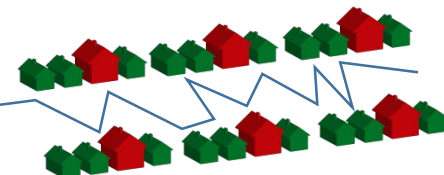
$$d_i^{RMS} = \sqrt{\frac{1}{M} \sum_{k=0}^{M-1} d_{ik}^2} = \frac{\sigma_i}{\sqrt{M}}$$



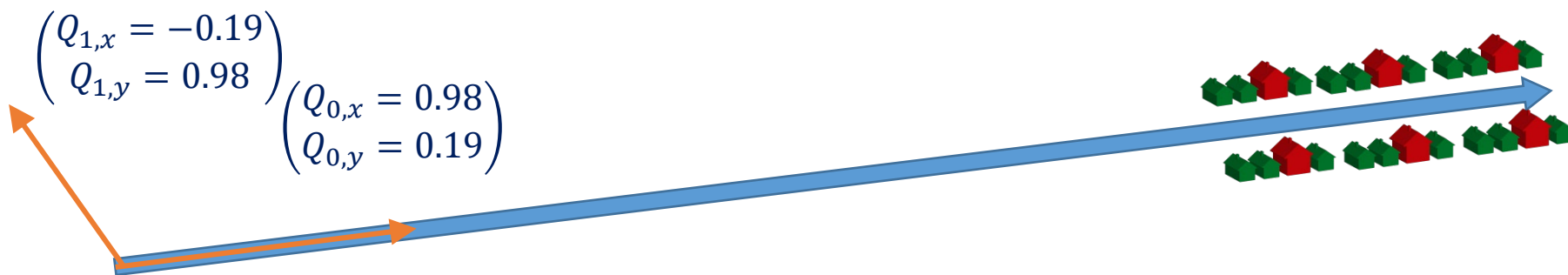
Often, the engineering context puts physical meaning on the singular values

Example: The delivery driver; the 2D locations of the destinations are the data

$$R = \begin{pmatrix} 9.8 & 10.1 & 9.7 & 10.2 \\ 2.1 & 2.1 & 1.8 & 1.9 \end{pmatrix}$$



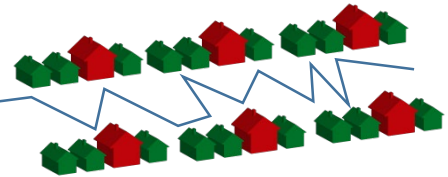
$$\begin{pmatrix} R \end{pmatrix}^{2 \times 4} = \begin{pmatrix} Q \end{pmatrix}^{2 \times 2} \begin{pmatrix} 20.3 \\ 0.25 \end{pmatrix}^{2 \times 4} \begin{pmatrix} P^T \end{pmatrix}^{4 \times 4}$$



We are told that the most important direction is Q_0 because the measure of the distance in that direction is ~ 100 times more than in the Q_1 direction

Example: The delivery driver; the 2D locations of the destinations are the data

$$R = \begin{pmatrix} 9.8 & 10.1 & 9.7 & 10.2 \\ 2.1 & 2.1 & 1.8 & 1.9 \end{pmatrix}$$



$$\begin{pmatrix} R \end{pmatrix}^{2 \times 4} = \begin{pmatrix} Q \end{pmatrix}^{2 \times 2} \begin{pmatrix} 20.3 \\ 0.25 \end{pmatrix}^{2 \times 4} \begin{pmatrix} P^T \end{pmatrix}^{4 \times 4}$$

Note that with the specific singular value 20.3 and $d_i^{RMS} = \frac{\sigma_i}{\sqrt{M}}$

the RMS distance to the destinations along the *dominant* direction is

$$d_0^{RMS} = \frac{20.3}{2} = 10.15$$

SVD in Signal Processing

The geometrical picture allows to think of a Signal + Noise in this manner

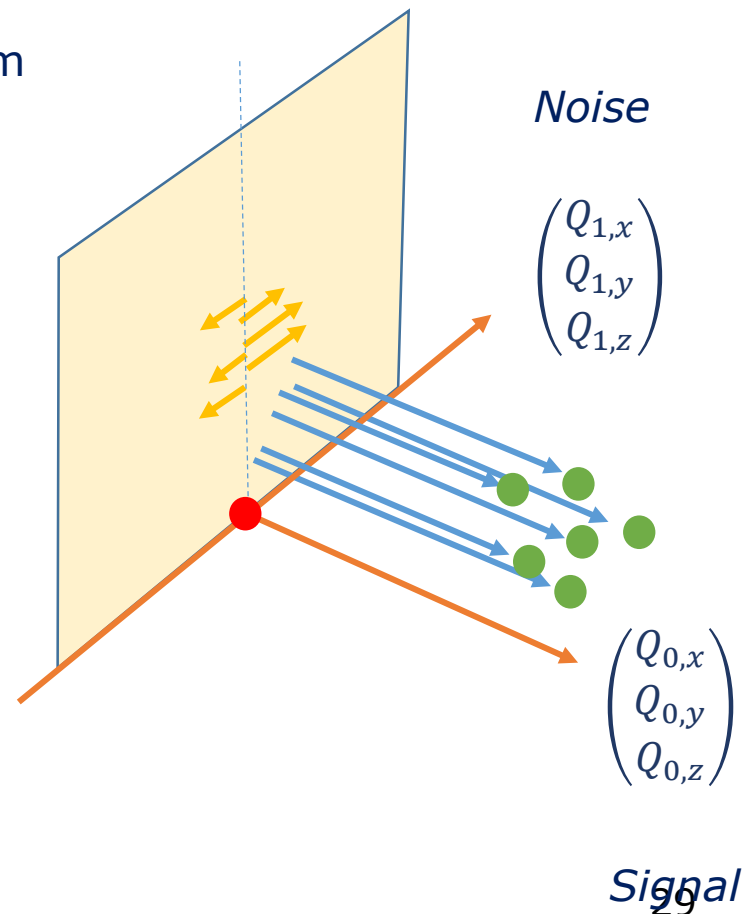
Example: Noise - random variations about an otherwise predictable waveform

SVD is separating the "common behaviour" from the variations visualised "pictorially" as shown

SVD - a really powerful tool for denoising data

More on this in the Signal Processing lectures of this module

A topical example is ...



SVD in Communications

Example: Multiple "versions" of the same wireless signal are received by different antennas pointing in different directions

We might simplify; each antenna results in a column of data values measured at sequential time instants

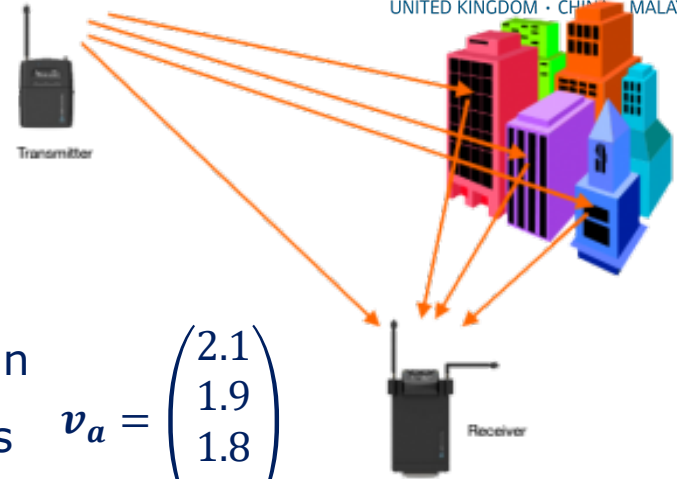
$$v_a = \begin{pmatrix} 2.1 \\ 1.9 \\ 1.8 \\ 1.7 \end{pmatrix}$$

Put these all together in a rectangular matrix $\begin{pmatrix} v_a & v_b & \dots \end{pmatrix}$

SVD of this, $Q\sigma P^T$, reveals that what these sets *mostly* have in common is Q_0 and the size of σ_0 compared to the other singular values indicates how much variation there is between the data received by the different antennas

Note If all signals were identical, then $\sigma_0 \neq 0$ and all other singular values are 0

Improved immunity to multipath and interference effects



Example: A LTI system has a number of parameters that describe what *state* it is in. The state is representable as a column of values, $\mathbf{s}$.

As time evolves, we record an output(s) as elements of a column vector, $\mathbf{y}$. Also we have an input that we can change as time evolves, $\mathbf{x}$.

Is it possible to deduce the state of the system, $\mathbf{s}$, by making a sequence of measurements, $\mathbf{y}$? **OBSERVABILITY**

Is it possible to change system to any state, $\mathbf{s}$, we wish by applying a particular sequence of input values, $\mathbf{x}$? **CONTROLLABILITY**

As we have an LTI we can legitimately express these as $\mathbf{y} = \mathbf{A}\mathbf{s}$ $\mathbf{s} = \mathbf{B}\mathbf{x}$

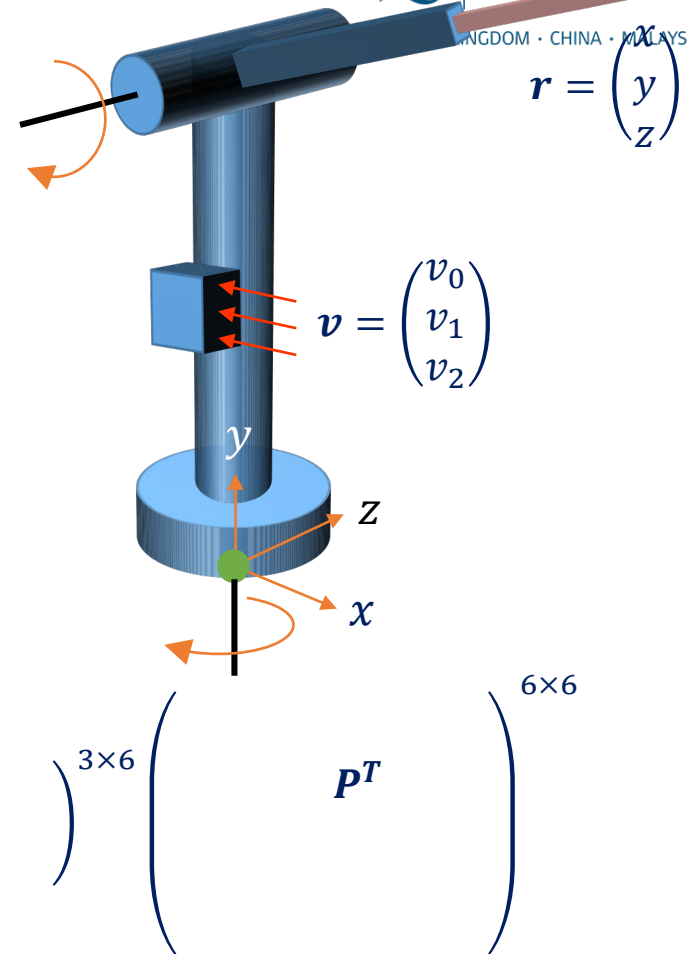
SVD reveals, via the true rank - how rectangular the A and B matrices are

Geometrical Interpretation of Column Vector Data

Example: The broken robotic arm

$$r_a = \begin{pmatrix} 2 \\ 1 \\ 0 \end{pmatrix} \quad r_b = \begin{pmatrix} 3 \\ -1 \\ 0 \end{pmatrix} \quad r_c = \begin{pmatrix} 1 \\ 5 \\ 0 \end{pmatrix} \quad r_d = \begin{pmatrix} 3 \\ 3 \\ 0 \end{pmatrix}$$

$$r_e = \begin{pmatrix} 7 \\ 1 \\ 0 \end{pmatrix} \quad r_f = \begin{pmatrix} 4 \\ 2 \\ 0 \end{pmatrix}$$



$$\begin{pmatrix} R \end{pmatrix}^{3 \times 6} = \begin{pmatrix} Q_{0,x} & Q_{1,x} & 0 \\ Q_{0,y} & Q_{1,y} & 0 \\ 0 & 0 & 1 \end{pmatrix}^{3 \times 3} \begin{pmatrix} 10.1 & & \\ & 5.29 & \\ & & 0 \end{pmatrix}^{3 \times 6} \begin{pmatrix} P^T \end{pmatrix}^{6 \times 6}$$

We see that SVD tells us that there is *zero movement along one of the axes*,

$Q_2 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$. We do get 3 axes vectors, because it really is a 3D problem, but because

$\sigma_0 = 0$, SVD is telling us that *our target data didn't ever use one of the dimensions*

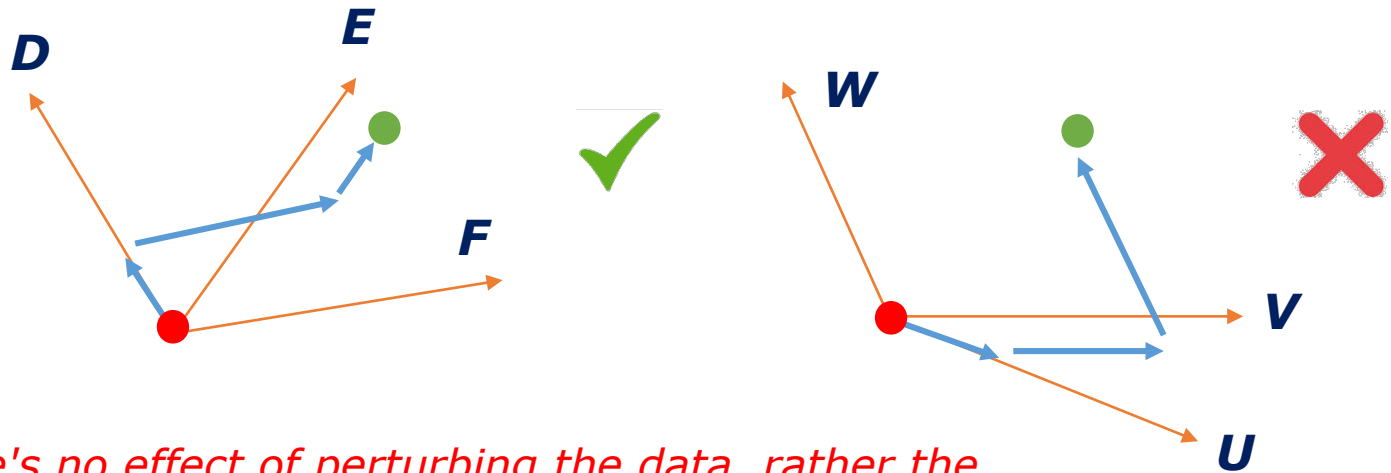
The axes used by SVD are *perpendicular*



Imagine that we are slightly unsure of some of the data,
for example the exact position of those delivery destinations

Wouldn't it be nice if we had a degree of immunity to "slight perturbations" in the
data when calculating the fuel requirement

That is exactly what one gets if the axes used to represent each location are
perpendicular



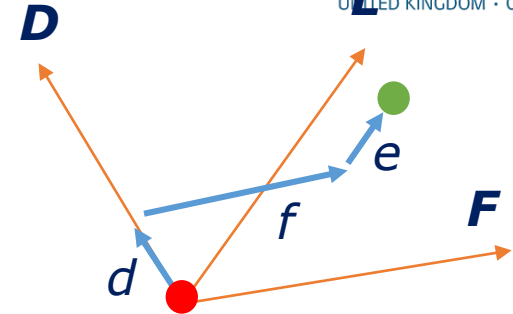
It's not that there's no effect of perturbing the data, rather the
use of perpendicular axes **minimises** the effect - useful!

How much of?

In our visualisation, we had sentences such as

This is expressing the destination as

"so much D " + "so much E " + "so much F "



How do we evaluate the "amount of distance" in each direction axis?

This is a practical attraction of orthogonal axes

Example: Given an SVD of a matrix $A^{N \times M} = Q^{N \times N} \sigma^{N \times M} P^T (M \times M)$ express the vectors $b^{N \times 1}$ and $x^{M \times 1}$ using the basis vectors provided by the SVD

Solution:

$$b = Q (Q^T b) \quad x = P (P^T x)$$

A column with elements giving how much of each of the Q basis vectors is required to represent b

We are not covering this in this brief course

The learning Objectives on this material concern its definition and properties and use

In an exam, you may be given a numerical example of an SVD and asked to use it to calculate an answer; See Self Study Sheet#4 for illustrations

Just wanted to be ultra clear on this!

Solving Matrix Problems using SVD



Example: Consider how the SVD representation can be used to solve $Ax = b$

Solution: Using $A = Q\sigma P^T$, recalling that $P^T P = P P^T = I$ and $Q^T Q = Q Q^T = I$

$$Ax = Q\sigma P^T x = b \quad Q\sigma P^T x = P^T b \quad \sigma P^T x = Q^T b \quad \begin{pmatrix} \sigma_0 & & \\ & \sigma_1 & \\ & & \dots \end{pmatrix} Q^T b = P^T x$$

A column with elements giving how much of each of the Q basis vectors is required to represent b

A column with elements giving how much of each of the P basis vectors is required to represent x

In words:

The amount of P_0 in unknown x is σ_0 times the amount of Q_0 in known b which is given by $Q_0^T b$

The amount of P_1 in unknown x is σ_1 times the amount of Q_1 in known b which is given by $Q_1^T b$

...

The amount of P_i in unknown x is σ_i times the amount of Q_i in known b which is given by $Q_i^T b$

Useful to see a small numerical example

Solving Matrix Problems using SVD



Example: Solve $Ax = b$ given that the SVD of $A = \begin{pmatrix} 1.24 & 1.42 \\ 0.37 & 0.58 \end{pmatrix}$ is known to be

$$A = \begin{pmatrix} 0.940 & -0.342 \\ 0.342 & 0.940 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 0.1 \end{pmatrix} \begin{pmatrix} 0.643 & 0.766 \\ -0.766 & 0.643 \end{pmatrix} \quad \text{and} \quad b = \begin{pmatrix} 3 \\ -7 \end{pmatrix}$$

Solution:

$$A = Q\sigma P^T$$

$$\sigma P^T x = Q^T b$$

Evaluate the amounts of each Q basis vector used to represent b

$$Q^T b = \begin{pmatrix} 0.940 & 0.342 \\ -0.342 & 0.940 \end{pmatrix} \begin{pmatrix} 3 \\ -7 \end{pmatrix} = \begin{pmatrix} 0.43 \\ -7.61 \end{pmatrix}$$

$$x = P (P^T x) = P (\sigma^{-1} Q^T b) = P \begin{pmatrix} 0.5 & 0 \\ 0 & 10 \end{pmatrix} \begin{pmatrix} 0.43 \\ -7.61 \end{pmatrix} = P \begin{pmatrix} 0.21 \\ -76.1 \end{pmatrix}$$

$$x = \begin{pmatrix} 0.643 & -0.766 \\ 0.766 & 0.643 \end{pmatrix} \begin{pmatrix} 0.21 \\ -76.1 \end{pmatrix} = \begin{pmatrix} 58.4 \\ -48.8 \end{pmatrix}$$

Solving Matrix Problems using SVD

Earlier I claimed that: *SVD resolves many many numerical computation problems - for example if a matrix solve problem is on the "borderline" because of finite precision issues*

Example: Solve $Ax = b$ with $A = \begin{pmatrix} 1 & 2 & 4 \\ 2 & 3 & 5 \\ 3 & 5 & 9 \end{pmatrix}$ using SVD

Solution: Do we spot that the 3rd row is the sum of the other 2 - therefore we do not have 3 "independent equations" which we know means this is not *uniquely* solvable. How do we see this with SVD?

The SVD of A is $A = Q \begin{pmatrix} -0.35 & 0.74 & -0.58 \\ -0.47 & -0.67 & -0.58 \\ -0.81 & 0.07 & 0.58 \end{pmatrix} \begin{pmatrix} 13.2 & & \\ & 0.49 & \\ & & 1.16e^{-16} \end{pmatrix} \begin{pmatrix} -0.28 & -0.47 & -0.83 \\ -0.80 & -0.37 & 0.48 \\ -0.53 & 0.80 & -0.27 \end{pmatrix}$

The $1.16e^{-16}$ is what the computer gives for 0

One or more zero singular values - or virtually zero in real computations indicates that the system of equations does not have a unique solution

Solving Matrix Problems using SVD

You may have noticed how similar the *Eigenvalue* and *SVD* approaches are

The **Eigen Decomposition Theorem** for the real symmetric case

$$A = X \begin{pmatrix} k_0 & & \\ & k_1 & \\ & & \dots \end{pmatrix} X^T \quad \text{with } X^T X = I$$

It can be shown that the extension to non-symmetric and complex matrices is

$$A = X \begin{pmatrix} k_0 & & \\ & k_1 & \\ & & \dots \end{pmatrix} X^{-1} \quad \text{but } X^T X \neq I$$

Valid for any square matrix

Complex eigenvalues and eigenvectors

The **Singular Value Decomposition**

$$A = Q \begin{pmatrix} \sigma_0 & & \\ & \sigma_1 & \\ & & \dots \end{pmatrix} P^T \quad \text{with } Q^T Q = I \quad \text{and } P^T P = I$$

Valid for any generally rectangular matrix The complex case needs a tweak to this.

Real singular values and generally complex singular vectors

Both have their valuable uses and pros and cons

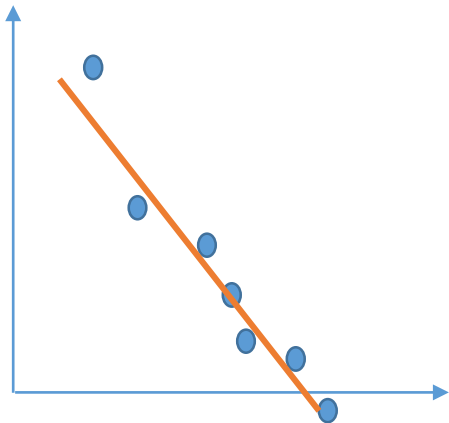
Rectangular Matrix Problems with SVD

The rectangular cases for $Ax = b$ are typified by

$$\begin{pmatrix} \sigma_0 & 0 & 0 & 0 \\ 0 & \sigma_1 & 0 & 0 \end{pmatrix} \begin{pmatrix} Q_0^T b \\ Q_1^T b \\ Q_2^T b \\ Q_3^T b \end{pmatrix} = \begin{pmatrix} P_0^T x \\ P_1^T x \end{pmatrix}$$

More equations than unknowns -
over specified

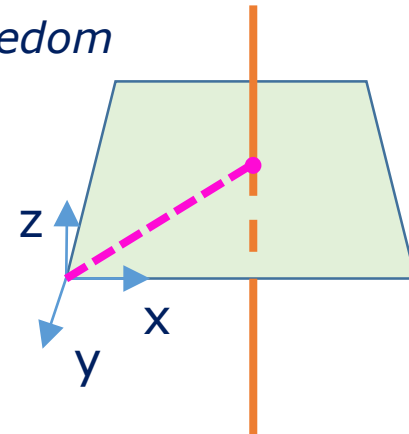
Typical of *best fit* problems



$$\begin{pmatrix} \sigma_0 & 0 \\ 0 & \sigma_1 \\ 0 & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} Q_0^T b \\ Q_1^T b \end{pmatrix} = \begin{pmatrix} P_0^T x \\ P_1^T x \\ P_2^T x \\ P_3^T x \end{pmatrix}$$

More unknowns than equations -
under specified

*Design is constrained, but still
some freedom*



Given our time constraints, we'll just explore the first

Rectangular Matrix Problems with SVD

Example: Complexity reduction. Let us scan a grey scale image. There are M by N pixels each of which has a grey value in the range 0-255

Solution: Create an N by M matrix of the pixel values such as $V = \begin{pmatrix} 150 & 200 & 180 \\ 77 & 202 & 100 \\ 127 & 200 & 98 \end{pmatrix}$

Store or transmit **$M*N$** 8 bit values

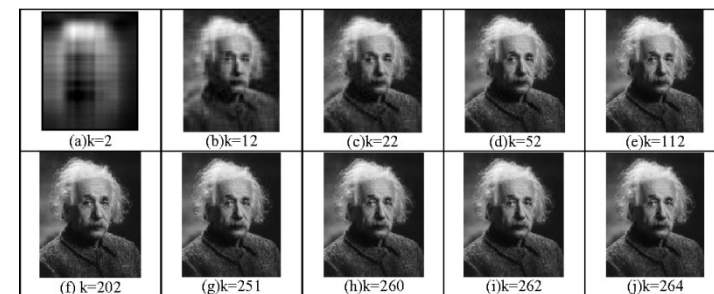
$$\text{SVD gives } V = \begin{pmatrix} -0.66 & 0.73 & -0.19 \\ -0.51 & -0.62 & -0.60 \\ -0.55 & -0.30 & 0.78 \end{pmatrix} \begin{pmatrix} 461.3 & & \\ & 60.6 & \\ & & 30.2 \end{pmatrix} \begin{pmatrix} -0.45 & -0.75 & -0.49 \\ 23 & -0.65 & 0.65 \\ 0.81 & -0.109 & -0.58 \end{pmatrix}$$

Remove least significant aspects progressively

$$\begin{pmatrix} -0.66 & 0.73 \\ -0.51 & -0.62 \\ -0.55 & -0.30 \end{pmatrix} \begin{pmatrix} 461.3 & \\ & 60.6 \end{pmatrix} \begin{pmatrix} -0.45 & -0.75 & -0.49 \\ 23 & -0.65 & 0.65 \end{pmatrix}$$

noting the the extreme case of

$$\begin{pmatrix} -0.66 \\ -0.51 \\ -0.55 \end{pmatrix} \begin{pmatrix} 461.3 \end{pmatrix} \begin{pmatrix} -0.45 & -0.75 & -0.49 \end{pmatrix}$$



Store or transmit **$M+N+1$** values
(encode decimals as 8 bit integers?)